Agents, MCP, and Where FL-04 Fits

Name: Sibgha Mursaleen
Date: 19 July 2026

What an agent actually is

The word "agent" gets used for almost anything with an LLM in it, which makes it close to meaningless without a stricter definition. Anthropic draws one useful line: it's not about how many LLM calls a system makes, or how impressive the output looks. It's about who decides what happens next.

In a system where a developer writes the sequence of steps call the model, pass the output to the next step, call the model again the code is in control. The model just fills in content at each fixed point. That's still useful, but it's not an agent.

An agent is different: the model itself decides what to do next, based on what just happened. It picks which tool to call, evaluates the result, and decides whether to try again, ask for more information, or stop. The control loop lives inside the model's reasoning, not in a diagram you drew in advance. That's also why agents cost more and are riskier to run unsupervised nobody pre-approved the exact sequence of actions, so mistakes can compound before a human sees them.

What MCP is

MCP (Model Context Protocol) solves a separate but related problem: how does a model actually reach outside the chat window and touch something real a file, a database, a live API without every developer writing custom glue code for every service?

MCP standardizes that connection. A server exposes what it can offer through three types of building blocks:

- **Tools**: things the model can *do*. Send an email, run a query, create a file. These have side effects.
- **Resources**: things the model can *read*. File contents, database records, page content. No side effects, just context.
- **Prompts**: reusable instruction templates a server provides, so common tasks don't need to be re-written from scratch each time.

A client (like Claude) connects to a server, discovers what tools/resources/prompts it offers, and calls them as needed. This is what makes agentic behavior possible in practice an agent needs something to act *through*, and MCP is one standard way of giving it that.

MCP does not make a system an agent by itself. Instead, it provides a standard way for AI applications to access external tools, resources, and prompts. Agents often use MCP because it gives them a consistent way to interact with outside systems.

Workflow vs. agent, applied to FL-04

My FL-04 pipeline a five-stage n8n pipeline that reads a technical article and produces a study report is a **workflow**, not an agent. Specifically, it's the **prompt chaining** pattern: a fixed sequence of LLM calls where each stage's output feeds the next, with a gate in between.

Tracing the actual node graph confirms this. The connection map is:

Manual Trigger → Edit Fields → Stage 1 (Summarize) → Wait/Form (human gate) → Stage 2 (AI Review) → Stage 3 (Final Report)

Every node has exactly one outgoing edge to a fixed next node. There's no branching, no loop-back, and no point where the model chooses which node runs next or which tool to invoke. Each chainLlm node is a single augmented LLM call filling in content within a slot I defined at design time. The Wait/Form node acts as the human checkpoint the "gate" in Anthropic's chaining diagram but it's a person approving or rejecting, not the model directing its own path.

Where the workflow breaks, and what an agent upgrade would fix

Testing FL-04 against five real sources surfaced a specific, traceable problem: neither Stage 2 (AI review) nor the human reviewer ever sees the original source article. Both only see the summary text. On two of five runs, this let fabricated or off-topic content pass both checkpoints at "High confidence" including a summary of the Model Context Protocol that described an entirely different concept. The workflow executed exactly as designed every time; the failure is a structural gap, not a bug.

The concrete upgrade this points to: give Stage 2 access to a tool that retrieves the original article, then allow the model to decide whether the summary is sufficiently grounded. If unsupported claims are detected, the model should choose to regenerate the summary, request human review, or continue to the final report. This makes the next action depend on the model's evaluation rather than a fixed sequence of nodes.